

Searching Binary Embeddings

A Brief Comparative Study

12 September 2026

Abstract

This study compares three methods for searching binary document embeddings: scanning selected clusters, splitting candidates with bitplanes, and looking up short bit patterns in increasing mismatch cost. Each method uses the same score and is evaluated under a recall requirement and memory limit. On one million real passage embeddings, neither alternative established a speed advantage over the tuned scan. Constructed inputs show a small key-lookup advantage when the important coordinates remain fixed and a large bitplane advantage when they change by query. The cost analysis explains these outcomes and the limits of extrapolating them to larger collections.

1 Introduction

Exa’s public vector-database description provides the starting point: compact binary document vectors, floating-point queries, and fast scoring inside selected clusters [1]. The question examined here is whether information in the query can reduce the work inside those clusters. Method A scans every selected document. Method B divides the candidates by individual bits. Method C begins with a short bit pattern preferred by the query and checks alternatives.

The objective is the lowest query time at a specified recall and RAM limit. Recall measures how many of the correct reference results are returned. Avoiding document scores is useful only when the additional selection work costs less than the scores it avoids.

1.1 One query, six documents

A stored 1 represents +1; a stored 0 represents −1. For the floating-point query $q = [0.7, 0.5, -0.4, -0.6, 0.3]$, the preferred sign pattern is 11001. Its score is $Q = \sum_i |q_i| = 2.5$. A mismatch at coordinate i changes a contribution from $+|q_i|$ to $-|q_i|$. The score therefore falls by $2|q_i|$:

$$S(q, b) = Q - 2p, \quad p = \sum_{i: b_i \neq \text{sign}(q_i)} |q_i|. \quad (1)$$

Here b_i is the decoded document sign. For example, 11000 mismatches only the last coordinate, so its score is $2.5 - 2(0.3) = 1.9$.

ID	Document signs	First-bit cluster	Score
0	01001	0	1.1
1	00100	0	−1.3
2	10100	1	0.1
3	11001	1	2.5
4	11000	1	1.9
5	00011	0	−1.1

The correct top two are IDs 3 and 4. All methods use the same score and break ties by the smaller ID. The following examples open both clusters, so routing loses no document.

2 Full scan

Before a query arrives, the index stores each document’s packed signs, ID, and cluster assignment. A cluster is a stored subset of documents. A query first selects clusters, then Method A reads and scores every document in them.

Before queries: text $\rightarrow$ embeddings $\rightarrow$ packed signs $\rightarrow$ index
Each query: embedding $\rightarrow$ select clusters $\rightarrow$ A, B, or C $\rightarrow$ top K

Opening one cluster is one *probe*. With N documents, C clusters, and P probes, balanced clusters give approximately NP/C selected rows. Actual cluster sizes can differ, so the correct count is $F_A = \sum_{c \in \mathcal{P}} n_c$, where $\mathcal{P}$ contains the selected clusters. Setting $C = P = 1$ removes routing and searches the entire collection.

The experiments include IVF, which searches selected learned clusters, and fixed sign-prefix groups. A k -bit prefix defines up to 2^k groups. These are different routing choices; neither is assumed to reproduce Exa’s complete production system.

Listing 1: Method A: score every selected document.

```
scan(query, clusters, K):
    table = prepare_score_table(query)
    best = empty_top_k(K)
    for cluster in clusters:
        for document in cluster.documents:
            score = full_score(table, document.code)
            best.offer(score, document.id)
    return best.sorted_by_score_then_id()
```

In the six-document example, A scores all six rows and retains IDs 3 and 4. The **best** structure keeps only the current top K . It is implemented as a heap: a small structure that makes the worst retained result quick to find and replace.

2.1 Scoring cost

The scorer uses the lookup-table technique described by Exa [1]. For each group of g coordinates, it prepares the contributions of all 2^g sign patterns. A document then needs $G = \lceil d/g \rceil$ table terms, where d is the code length. The measured implementation uses $g = 4$: a 256-bit document needs 64 terms. Each term includes extracting bits, reading a table value, and adding it; it is not a single CPU instruction.

$$T_A^{\text{local}} \approx T_{\text{table}} + F_A c_{\text{scan}} + T_{\text{select}}(F_A, K). \quad (2)$$

Here c_{scan} is the measured time to read and score a consecutive row. The final term is the time spent retaining the best results. If a measured row cost already includes that selection, the selection term must not be added again.

Matryoshka training supports useful shorter embedding prefixes [1, 2]. It does not sort the coordinates of each query by magnitude or guarantee the recall of 24 sign bits. The resulting document representation is held fixed across A, B, and C in each comparison.

3 Bitplane search

A *bitplane* stores one coordinate across all documents in a cluster. For cluster 1, the local row order is [2, 3, 4], with signs 10100, 11001, and 11000. Coordinate 2 forms plane 011; coordinate 3 forms 100. A candidate mask has one bit per local document and records which rows remain in a branch.

For a positive query coordinate, splitting parent mask A by plane B_i creates $A \& B_i$ and $A \& \neg B_i$. The first child matches the query sign; the other adds penalty $|q_i|$. For a negative query coordinate, the roles are reversed. Method B considers larger query magnitudes first because a mismatch there reduces the possible score more strongly.

The other child remains in a priority queue: a collection ordered by the lowest known penalty. A branch with penalty p cannot score above $Q - 2p$, even if all remaining bits match. This *upper bound* allows a branch to be discarded when it cannot improve the current results. In cluster 1, splitting coordinate 2 separates [3, 4] from [2]. After scoring [3, 4], the other branch has bound $2.5 - 2(0.5) = 1.5$, below the second-best score 1.9.

Listing 2: Method B: preserve alternatives and discard only justified branches.

```
branch(query, clusters, K, leaf_size, node_budget):
    table = prepare_score_table(query)
    order = coordinates_by_decreasing_abs(query)
    queue = full candidate mask for each selected cluster
    best = empty_top_k(K)
    while queue is not empty:
        node = remove_lowest_penalty(queue)
        if cannot_improve(node, query, best): continue
        if work_limit_reached(node_budget): break
        if node.count <= leaf_size or no_bits_remain(node):
            score_set_documents(table, node.mask, best)
        else:
            queue_nonempty_children(split(node, order))
    return best.sorted_by_score_then_id()
```

A final group scored without further splitting is a *leaf*; L is its size threshold. A work limit can stop too early and lose recall. Without such a limit, safe bounds and the ID tie rule allow exact search within the selected clusters.

3.1 The full-width cost

With w bits per machine word, every dense mask in cluster c spans $W_c = \lceil n_c/w \rceil$ words. It keeps that width even after most candidates are removed. For 6,400 documents and 64-bit words, each mask spans 100 words. Reducing the candidates to 80 does not reduce the width; ten further splits still visit 1,000 words.

If J_c counts splits and E_c counts enumerated leaf masks, the word visits are $V_s = \sum_c J_c W_c$ and $V_l = \sum_c E_c W_c$, over selected clusters. Thus

$$T_B^{\text{local}} \approx T_{\text{table}} + T_{\text{order}} + V_s c_{\text{split}} + V_l c_{\text{leaf}} + F_B c_{\text{gather}} + T_{\text{queue}} + T_{\text{select}}(F_B, K). \quad (3)$$

The F_B scored rows may be scattered in memory. Their measured cost can exceed the consecutive-row cost in A. Leaf-word visits, mask allocation, and queue work also remain.

4 Key lookup

Method C implements the proposal to begin with the query’s preferred pattern and consider less favorable patterns. A fixed set of h coordinates forms a short *key*. Within each cluster, documents with the same key are stored together. A directory maps each occupied key to a starting position and row count. The packed document codes remain available for full scoring.

In the example, select coordinates 2 and 3. The query prefers key 10. It finds document 0 in cluster 0 and documents 3 and 4 in cluster 1:

Key	Penalty	Cluster 0 IDs	Cluster 1 IDs
10	0	[0]	[3, 4]
11	0.4	[]	[]
00	0.5	[5]	[]
01	0.9	[1]	[2]

The order depends on the sum of magnitudes at flipped coordinates. It is not simply the number of flipped bits. After scoring the first key’s three documents, the current second score is 1.9. Every remaining key has penalty at least 0.4, so its best possible full score is at most 1.7. C can stop without checking the empty 11 key.

Listing 3: Method C: enumerate keys by weighted penalty.

```
key_search(query, clusters, K, key_bits, limits):
    table = prepare_score_table(query)
    keys = unique_keys_by_increasing_penalty(query, key_bits)
    best = empty_top_k(K)
    for key, penalty in keys:
        if cannot_improve(penalty, query, best): break
        for cluster in clusters:
            if work_limit_reached(limits):
                return best.sorted_by_score_then_id()
            rows = cluster.directory.lookup(key)
            score_each_row(table, rows, best)
    return best.sorted_by_score_then_id()
```

There is one shared key-order queue across selected clusters. Each key is looked up in each cluster. Empty lookups still take time. If E complete keys are tried across P clusters, there are $H = PE$ lookups; a limit that stops partway through a key requires the actual count instead.

4.1 Cost and stopping

If F_C rows receive full scores, the local time is approximately

$$T_C^{\text{local}} \approx T_{\text{table}} + T_{\text{key order}} + T_{\text{key queue}} + Hc_{\text{lookup}} + F_C c_{\text{posting}} + T_{\text{select}}(F_C, K). \quad (4)$$

Here a posting is a document ID found through the directory; c_{posting} includes fetching and scoring its row. The cost of enumerating keys cannot be replaced by one constant-time lookup, and the current implementation still scores every returned candidate.

A short key may omit coordinates that determine the full ranking. Finding many documents therefore does not establish high recall. Stopping after a candidate quota is approximate; stopping after all remaining score bounds fail, or after exhaustion, is exact within the selected clusters. Ties and floating-point rounding must be handled consistently with A. Like B, C cannot recover a document from a cluster that was never opened.

5 Memory and parameter selection

The memory calculation separates stored data from temporary query work. A packed word with w bits takes $u = w/8$ bytes. With an ID width of b_{id} bytes, the row data occupies

$$M_{\text{rows}} = N(u \lceil d/w \rceil + b_{\text{id}}). \quad (5)$$

At $d = 256$, $w = 64$, and eight-byte IDs, this is 40 bytes per document. It excludes reserved array capacity and supporting structures. For example, six eight-byte IDs contain 48 bytes, but an array with capacity for eight uses 64 bytes.

B retains the rows and adds bitplanes. Their stored data size is

$$M_{\text{planes}} = du \sum_{c=1}^C \lceil n_c/w \rceil. \quad (6)$$

For one large cluster, this is approximately $Nd/8$ extra bytes. Small clusters introduce padding to whole words. During a query, B also holds masks for waiting branches. If $A_c(t)$ is the number of allocated masks for cluster c at time t , their peak data size is $u \max_t \sum_c A_c(t) W_c$. Queue records are additional.

C reorders the existing rows and IDs, so those IDs also serve as postings; it does not need a second copy of all IDs. Its directory has a key, start position, and count per occupied key. For four-byte keys and eight-byte start/count fields, these fields total 20 bytes. The C++ hash table allocates more for nodes, pointers, and its bucket array. Twenty bytes is a field total, not an allocated-memory estimate.

Let M_0 include allocated rows, routing data, and common containers. Let D_C be C's directory allocation, and Q_0 the common query data: the query, score table, cluster IDs, and top- K results. Its main size terms are $O(d + G2^g + P + K)$.

Method	Stored index	Temporary query memory
A	M_0	Q_0
B	$M_0 + M_{\text{planes,allocated}}$	$Q_0 + \text{masks and branch queue}$
C	$M_0 + D_C$	$Q_0 + \text{key queue}$

Total serving RAM also includes the program, retained inputs, and simultaneously existing Python/C++ outputs. Building the index can require more memory because old inputs and the new index coexist. Actual M_0 may differ between methods because their containers reserve different capacities.

5.1 Choosing a useful split

Cluster count C , probes P , code length d , and requested results K affect all methods. B also has leaf size and node budget; C has key width, lookup budget, and candidate budget. Random branch exploration changes visit order and requires repeated seeds when tested.

Under a balanced, single-path approximation with j cuts, the remaining rows number $N2^{-j}$. One more cut saves approximately $N2^{-(j+1)}$ scores. It is useful only if

$$N2^{-(j+1)} c_{\text{score}} > W c_{\text{split}} + \Delta T_{\text{queue}}. \quad (7)$$

This condition suggests depths to test. It does not establish recall or account for every branch. Since $W = \lceil N/w \rceil$ also grows with N , a larger collection alone does not guarantee that B becomes faster. Each method must be tuned under the same memory and recall constraints.

6 Recall

Recall must name its reference ranking. If $R_K(q)$ is the exact top- K set and $A_K(q)$ the returned set, then

$$\text{Recall@ } K(q) = \frac{|A_K(q) \cap R_K(q)|}{K}. \quad (8)$$

Returning $[3, 0]$ when the reference is $[3, 4]$ gives recall $1/2$. This study’s primary reference is the exact floating-query/binary-document score. Recovery of the original float-vector ranking, human relevance, and answer accuracy are separate measurements.

6.1 Where results are lost

For a fixed query, choose a document uniformly from its true top K . Let O mean that its cluster is opened and F that it is returned. Since an unopened document cannot be returned, recall satisfies the conditional identity

$$R(q) = \Pr(O \mid q) \Pr(F \mid O, q). \quad (9)$$

For example, if routing retains eight of ten reference documents and local search returns six of those eight, recall is $(8/10)(6/8) = 60\%$. This does not assume that routing and local search are independent. Across queries, average these products; multiplying the two separate averages generally gives a different answer.

6.2 A distribution with an exact calculation

A simple model explains when the preferred key is sufficient. Assume independent, equally likely document signs. Each query has m strong coordinates of magnitude a and $d - m$ weak coordinates of magnitude ϵ . Require $a > (d - m)\epsilon$. Then one strong mismatch costs more than every possible weak mismatch combined, so documents matching all strong signs outrank every document with a strong mismatch.

A document matches all m strong signs with probability 2^{-m} . The number Z of such documents follows a binomial distribution:

$$Z \sim \text{Binomial}(N, 2^{-m}), \quad \Pr(Z = z) = \binom{N}{z} (2^{-m})^z (1 - 2^{-m})^{N-z}. \quad (10)$$

If only this ideal group is searched, and its rows are fully scored, the returned reference count is $\min(Z, K)$. Hence

$$\mathbb{E}[R] = \frac{\mathbb{E}[\min(Z, K)]}{K} = \frac{1}{K} \sum_{i=1}^K \Pr(Z \geq i), \quad N \geq K. \quad (11)$$

For $N = 2$, $m = 1$, and $K = 1$, the ideal group is empty with probability $(1/2)^2 = 1/4$. Expected recall is therefore $3/4$, although the group contains one document on average. Replacing the random group size with its mean would incorrectly predict perfect recall.

This result applies when the searched group uses all strong coordinates. It does not assign recall to an arbitrary 24-bit key, budget-limited traversal, or learned cluster router. Real embeddings have dependent coordinates and different query weights. An earlier normal approximation in this study missed recall by up to 9.6 percentage points, beyond its two-point criterion. Measured recall remains necessary; an assumed embedding “accuracy” cannot substitute for the distribution of ranking errors.

7 Real-data results

The real-data study uses Nomic embeddings of MS MARCO passages [3, 4]. All methods receive the same 256-bit document codes and floating-point queries, return the top 100, and use the same C++ scorer. The reference is the exact binary ranking; the primary comparison includes no float reranker or human relevance labels.

The machine is an Apple M5 Max with 128 GiB of physical memory. Experiments use a separate 32 GB decimal worker budget, one CPU thread, and one query at a time. Fresh processes build and warm each configuration. Timings include routing, search, and the Python-to-C++ call, with query encoding cached. Process blocks are repeated; recorded power settings do not establish constant CPU frequency.

7.1 A fair parameter comparison

Each method receives its fastest tested configuration that meets the tuning recall rule, then is evaluated unchanged on different query IDs. The cautious rule requires a lower recall estimate from repeated samples of tuning queries to meet the target. This provides margin without guaranteeing recall on future queries.

The study combines 6,821 tuning settings, including intermediate probe counts and direct sign routing for A and large leaves for B. Evaluation uses 200 query IDs excluded from the preceding 300-query collection. Table 1 shows one-million-document results.

Table 1: Median routing-plus-search time. Each row has the same achieved recall across methods.

Target	A (ms)	B (ms)	C (ms)	Recall (%)
80%	0.675	0.699	0.751	82.53
90%	1.634	1.686	1.880	91.73
95%	3.360	3.430	3.706	95.47
99%	9.580	9.965	11.013	99.25

A speedup is supported only when both methods meet the recall target and the full paired 95% interval for the ratio of A’s time to the alternative’s time exceeds one. Resampling keeps method timings paired by query and includes whole process blocks; repeated timings are not counted as new independent queries. None of the real study’s 48 comparisons established a B or C speedup under this rule.

A further control used 100 new query IDs, excluding all 500 earlier IDs. It gave C a one-bit key, A’s routing layout, and unlimited work budgets, while keeping A and B’s choices fixed. Times were 9.541, 9.855, and 10.545 ms for A, B, and C. All achieved 98.95% recall and therefore missed the 99% requirement. This remains a failure to meet the target, rather than a value rounded into a pass.

7.2 From text to results

A separate check encoded 20 existing independent query texts with the local Nomic model on Apple’s MPS GPU backend, then routed and searched them. Three blocks and two repeats produced 120 requests per method. At 99.45% recall, total median times were 16.313 ms for A, 16.303 ms for B, and 18.912 ms for C. The small A/B difference does not establish a speedup. Totals were measured per request: adding stage medians would not generally give the total median. These text timings cannot be assigned to constructed vectors that the encoder did not produce.

8 Controlled results

Constructed inputs isolate a condition that real embeddings need not satisfy. Document signs are independent and balanced over 256 coordinates. Each query has thirteen strong coordinates of magnitude one; all others have magnitude 0.0001. These satisfy the separation condition in Section 6. In the fixed case, the strong positions are always the same. In the changing case, each query selects thirteen positions uniformly without replacement.

The final follow-up used 40 tuning queries and 20 separate evaluation queries, with three process blocks and three timing repeats. It tested neighboring key widths, cluster counts, and leaf sizes, then remeasured earlier choices. At one million documents and a 99% target:

Table 2: Controlled-input median retrieval times. All results reach 100% recall except A with changing positions, which reaches 99.95%.

Strong positions	A (ms)	B (ms)	C (ms)
Fixed	0.0140	0.0160	0.0128
Changing	24.9235	0.1446	14.6301

With fixed positions, A directly opens two of 16,384 sign clusters, while C uses one physical cluster and a 13-bit key. The A/C time ratio is 1.098, with a paired 95% interval of 1.066–1.147. Both select documents through the same strong signs. C performs key generation and scoring in one C++ call, while A uses separate routing and scanning calls. The approximately 1.25-microsecond advantage concerns this implementation of closely related candidate selection.

With changing positions, B can select important coordinates separately for each query. Its chosen configuration uses one cluster, unlimited traversal, and leaf size 160. A opens 255 of 256 learned clusters. The A/B time ratio is 172.3, with a paired 95% interval of 169.1–177.2. C uses a 14-bit key. Its median is lower than A’s, but its speedup interval includes one, so a C advantage is not established in this case.

The leaf size has a concrete basis. Under the model, the ideal 13-bit group has mean $10^6/2^{13} \approx 122$ documents and standard deviation about 11. A threshold of 128 can force a further split on a weak coordinate. A threshold of 160 leaves more room to score the group. This calculation suggested a setting to test; it does not prove that 160 is optimal for arbitrary embeddings.

8.1 Validation of the cost model

Independent score calculations provide the reference ranking. Across applicable controlled runs, 447 per-query predictions matched recorded work counts, including dense-word visits, scored rows, empty lookups, and tracked mask memory where applicable. Queries outside the formulas’ conditions remain in the evidence but are not counted as successful predictions.

Correct work counts do not guarantee correct time estimates. Scoring approximately 1,958 scattered rows in a 16-million-row buffer took 0.159 ms with reused candidates and 0.455 ms with changing candidates. Earlier fits underestimated larger-buffer times by up to 45.2%. A revised complete-call model, frozen before an eight-million-row check, predicted all six checked times within 7.8%. Those were prescribed model-check configurations, not the best-tuned method comparison. The check tests collection-size transfer under the specified workload; it is not an error guarantee for new queries or a billion documents.

9 Larger collections

9.1 Memory first

At a billion documents with 256-bit codes and eight-byte IDs, codes occupy 32 GB and IDs occupy 8 GB. B adds at least 32 GB of bitplanes. These stored bytes remain even if a query opens very few clusters. Table 3 distinguishes the minimum row/plane data from the fuller planning estimate.

Table 3: Billion-document memory estimates in decimal GB. These are forecasts.

Layout	Minimum data	Planned serving RAM
A, direct prefix	40	about 49
C, global 13-bit key	40	about 49
B, one global cluster	72	about 83
B, prefix route and large leaf	72	about 81

The planning estimates include reserved space and a configurable 1 GB program reserve; that reserve is an assumption. All current in-memory layouts fail a 32 GB limit. B also fails 64 GB, while the listed A/C cases are predicted to fit. All listed cases fit a 1 TB planning budget, but that says nothing about the larger machine’s memory speed.

9.2 When coordinates remain fixed

Under the strong-coordinate model, the ideal 13-bit group contains approximately $10^9/2^{13} = 122,070$ rows on average. Equation 11 accounts for the chance that fewer than the requested 100 exist. A can route directly to this group; C can look up its key. Both then score the same rows. The fitted model predicts roughly 3 ms for either. B can use the same routing and a large leaf, producing similar work with additional storage. The forecast cannot reliably rank a difference of a few percent between these choices.

9.3 When coordinates change

To preserve the constructed B path at a billion rows, the leaf threshold is scaled to 160,000. Its thirteen useful cuts retain full-width masks. At the last split, the specified path and queued alternatives occupy approximately

$$(13 + 2) \left\lceil 10^9/64 \right\rceil \times 8 = 1.875 \text{ GB}$$

of mask data, including a parent while its children are created. Other paths can require different amounts. The complete-call model predicts about 0.10 s for this B path and 25 s for a global scan. Doubling B’s variable cost gives about 0.20 s; this is a sensitivity calculation, not a confidence interval. No complete billion-document index was measured.

A must also be allowed to route. If it scans fraction f of the collection, its time is approximately $T_{\text{route}} + fNc_{\text{scan}}$. Even with routing treated as free, A needs to scan less than about 0.41% to beat the 0.10 s B forecast, *while meeting the recall target*. A sufficiently selective router can therefore change the conclusion.

C’s fixed key may miss the coordinates that matter. A fixed set of thirteen coordinates misses all thirteen randomly selected strong positions with probability $\binom{243}{13}/\binom{256}{13} \approx 49.94\%$. Finding a small or populated key group alone cannot establish that its documents preserve the ranking.

10 Conclusions and reproduction

The supported baseline on the evaluated real embeddings is A, the tuned scan. The controlled results identify two narrower cases: fixed important coordinates favor direct routing or key lookup; changing important coordinates can favor B’s query-dependent splits. The large B advantage was measured on constructed inputs. The small fixed-position C advantage was measured against a closely related candidate filter. Neither result establishes a universal advantage over IVF or Exa’s production system.

Equations 2–4 describe local search work. For one complete request, add the stages actually enabled:

$$T^{\text{request}} = T_{\text{embed}} + T_{\text{normalize}} + T_{\text{route}} + T^{\text{local}} + T_{\text{output/rerank}}.$$

Common filters, if present, must also be timed. A coefficient for one stage must not include work counted again in another. Similarly, peak serving RAM is the index plus all temporary allocations that coexist, including the running program. Stored data size alone is insufficient.

The practical procedure is to reject configurations that exceed RAM, use the cost formulas to identify promising settings, and compare each method’s fastest tested choice at the same recall target on separate evaluation queries. Work counts explain a result; complete-request measurements determine whether that result is useful. The evidence supports conclusions within the tested parameter range, rather than a proof of global optimality.

10.1 From the paper to the code

Paths below are relative to the companion repository. The detailed formal edition in **reports/search-math-formal/** retains the longer derivations and experimental record.

Purpose	Entry point
Run the six-document example	<code>examples/small_search.py</code>
Read the three implementations	<code>cpp/index.cpp</code> , <code>cpp/score.hpp</code> , <code>cpp/key_index.cpp</code>
Check work and recall models	<code>experiments/models.py</code> , <code>experiments/check_controlled.py</code>
Inspect the evidence	<code>reports/search-math/evidence/final-study.json</code> , <code>results/README.md</code>
Reproduce experiments	<code>experiments/README.md</code>

After the repository setup, run `.venv/bin/python examples/small_search.py` without downloading a corpus. Build this paper with `bash reports/search-math-brief-formal/build.sh`. The paper summarizes saved results; compiling it does not rerun experiments. The accompanying source archive rebuilds the PDF.

References

- [1] Exa Team. *How we built a web-scale vector database*. 2024. <https://exa.ai/blog/building-web-scale-vector-db>.
- [2] A. Kusupati et al. *Matryoshka Representation Learning*. 2022. <https://arxiv.org/abs/2205.13147>.
- [3] Microsoft. *MS MARCO*. Passage collection and benchmark resources. <https://microsoft.github.io/msmarco/>.
- [4] Nomic AI. *nomic-embed-text-v1.5*. Model documentation. <https://huggingface.co/nomic-ai/nomic-embed-text-v1.5>.